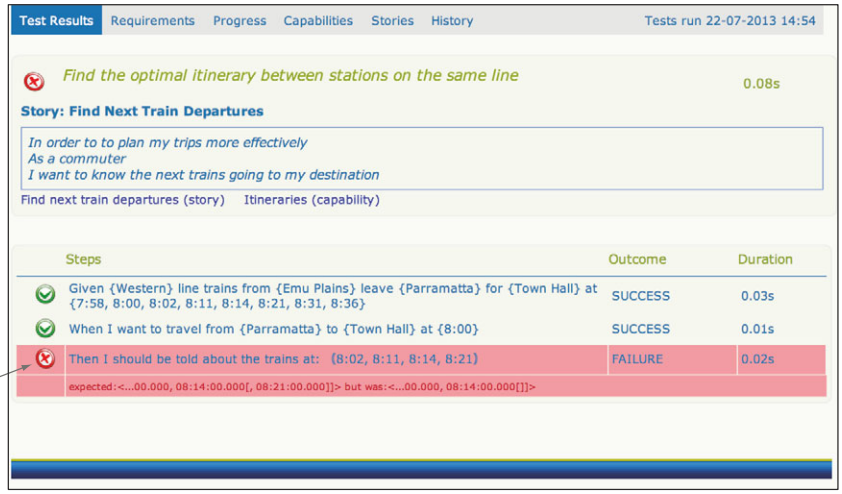


Figure 2.10 A failing acceptance criterion illustrates a difference between what the requirements ask for and what the application currently does.

This scenario expresses your current understanding of the requirement: the application currently behaves like this, and you have automated acceptance criteria and unit tests to prove it.

But the new user request has changed all this. The scenario now should be something like this:

You need more sample departure times.

```
Scenario: Find the optimal itinerary between stations on the same line
Given Western line trains from Emu Plains leave Parramatta for Town Hall
at 7:58, 8:00, 8:02, 8:11, 8:14, 8:21, 8:31, 8:36
When I want to travel from Parramatta to Town Hall at 8:00
Then I should be told about the trains at: 8:02, 8:11, 8:14, 8:21
```

You now want to know about all the trains in the next 30 minutes.

When you run this new scenario, it will fail (see figure 2.10). This is good! It demonstrates that the application doesn't do what the requirements ask of it. Now you have a starting point for implementing this modification.

From here, you can use the unit tests to isolate the code that needs to be changed. You'll update the "should calculate the correct arrival time" Spock specification to reflect the new acceptance criterion:

```
def "should calculate the correct arrival time"() {
  given:
    def westernLineFromEmuPlains =
        Line.named("Western").departingFrom("Emu Plains")

    timetableService.findLinesThrough("Parramatta","Town Hall") >>
                                            [westernLineFromEmuPlains]

    timetableService
           .findArrivalTimes(westernLineFromEmuPlains, "Parramatta") >>
                [at(7,58), at(8,00), at(8,02), at(8,11), at(8,14),
                 at(8,21), at(8,31), at(8,36)]
```

The pretend service now returns more trips.